Lab 13

Zigbee2MQTT: from the Zigbee Network to the MQTT Broker

Communication Technologies for the Internet of Things | Lecture 13

Aim and context

Lecture 13 integrates the Zigbee block, from Lecture 07, with the middleware block, from Lectures 11 and 12. In this lab, Zigbee2MQTT is installed on a Linux computer, a Zigbee device is paired and its state is observed arriving at a Mosquitto broker.

The work requires physical hardware: a CC2531 coordinator with the firmware already flashed, a Zigbee device and a Linux computer, which may be a Raspberry Pi. It is meant to be done during the class and is **not an assignment to hand in**. It also supports the development of the mini-project.

Note

Zigbee2MQTT evolves quickly and installation instructions change between versions. The commands below correspond to the official guide at the time of writing. Where they differ, the official guide prevails, at `https://www.zigbee2mqtt.io`. The results shown are examples.

Learning outcomes

- Install Mosquitto and Zigbee2MQTT and connect the USB coordinator.
- Configure the Zigbee network, with channel and join permission.
- Pair a device and identify it by its friendly name.
- Read state and command the device through MQTT topics.

Requirements

- A Linux computer with USB (Ubuntu or Raspberry Pi OS) and administrator access.
- A CC2531 coordinator with the `CC2531ZNP-Prod.hex` firmware already flashed, and a Zigbee device, for example a plug or a sensor.
- An Internet connection.
- Estimated duration: 60 to 90 minutes.

How to use this guide

Each step presents, in this order, the aim, the context, the theoretical background, the commands to run and the expected result. Read the whole step before running the first command. The boxes always have the same meaning:

Command to run

```
Command to run in the terminal.
```

Expected result (values may vary)

```
Result that should be displayed.
```

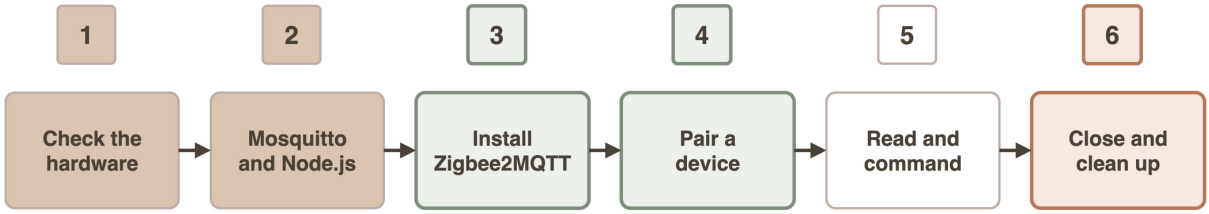
Analyse

A moment to pause and interpret the result, on your own or in discussion with colleagues.

Possible error and fix

An error that may occur and how to fix it.

Overview



Hardware: a CC2531 coordinator with firmware already flashed, a Zigbee device and a Raspberry Pi or Linux PC

How to read the figure

The six steps read from left to right. The first two prepare the computer. Steps 3 and 4, in green, install Zigbee2MQTT and pair the device. Step 5 reads state and sends commands over MQTT, and step 6 closes the network and cleans up.

Step 1 — Checking the hardware

Aim. Confirm that the computer recognises the USB coordinator.

Theoretical background

The CC2531 appears on Linux as a serial device, typically `/dev/ttyACM0`. Zigbee2MQTT needs this path, or a stable path under `/dev/serial/by-id`, which does not change when another USB device is plugged in.

Command to run

```
lsusb | grep -i 'texas'
ls -l /dev/serial/by-id/ /dev/ttyACM* 2>/dev/null
```

Expected result (values may vary)

```
Bus 001 Device 004: ID 0451:16a8 Texas Instruments, Inc. CC2531 USB CDC
crw-rw---- 1 root dialout 166, 0 /dev/ttyACM0
```

Possible error and fix

No device listed: check the coordinator in the USB port and try another port. **Permission denied** later: add the user to the `dialout` group with `sudo usermod -aG dialout $USER` and log in again.

Step 2 — Mosquitto and Node.js

Aim. Install the MQTT broker and the Zigbee2MQTT runtime.

Command to run

```
sudo apt update
sudo apt install -y mosquitto mosquitto-clients curl git make g++ gcc libsystemd-
    dev
sudo curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt install -y nodejs
sudo corepack enable
node --version
```

Expected result (values may vary)

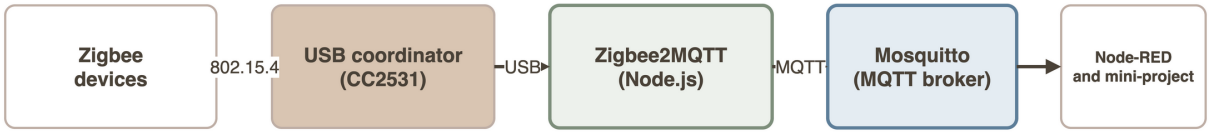
```
v22.x.x
```

Possible error and fix

node: command not found: the Node.js repository was not added. Repeat the `curl` command and the installation. Confirm the supported version in the official guide.

Step 3 — Installing Zigbee2MQTT

Aim. Get the code, install the dependencies and start the initial setup.



Open bridge: no manufacturer app or cloud

How to read the figure

The figure shows what ends up installed: the Zigbee2MQTT program receives the Zigbee network traffic from the USB coordinator and publishes it on Mosquitto. Node-RED, on the right, is the typical consumer.

Command to run

```
sudo mkdir /opt/zigbee2mqtt
sudo chown -R ${USER}: /opt/zigbee2mqtt
git clone --depth 1 https://github.com/Koenkk/zigbee2mqtt.git /opt/zigbee2mqtt
cd /opt/zigbee2mqtt
pnpm install --frozen-lockfile
pnpm start
```

On first run, open `http://<computer-address>:8080` in the browser. The initial setup page asks for the adapter, the broker address and the network channel.

Analyse

Which channel did you choose and why? Check, on the slide, which Zigbee channels stay away from Wi-Fi networks 1, 6 and 11.

Possible error and fix

Coordinator not found or a serial port error: give in the configuration the `/dev/serial/by-id` path confirmed in step 1. **Cannot connect to MQTT**: confirm that Mosquitto is active with `systemctl is-active mosquitto`.

Step 4 — Pairing a device

Aim. Allow a device to join, pair it and give it a name.



After pairing, joining should be closed

How to read the figure

The figure shows the five pairing steps. The first two are the operator's: allowing joining and putting the device in pairing mode. The interview and the name are done by Zigbee2MQTT. The last step, closing joining, is again the operator's.

In the web interface, enable device joining (*Permit join*) and put the device in pairing mode, as its manual says. In the program log, a sequence similar to the following appears:

Expected result (values may vary)

```
Zigbee2MQTT:info  Device '0x00124b0024c87aaf' joined
Zigbee2MQTT:info  Starting interview of '0x00124b0024c87aaf'
Zigbee2MQTT:info  Successfully interviewed '0x00124b0024c87aaf', device has successfully
    been paired
Zigbee2MQTT:info  Device '0x00124b0024c87aaf' is supported, identified as: SONOFF Zigbee
    smart plug (S26R2ZB)
```

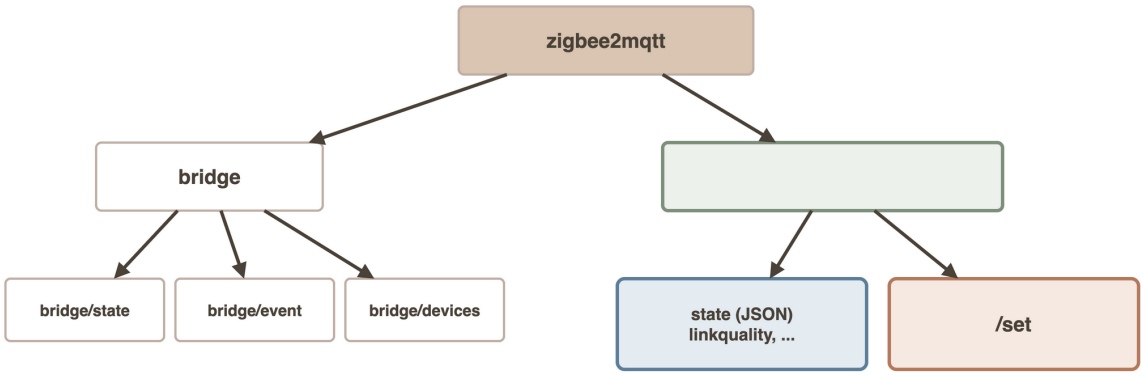
Give the device a friendly name, for example `tomada_sala`, in the web interface. Then turn off join permission.

Analyse

What information did the interview obtain? What distinguishes a supported device from an unsupported one? Why is join permission turned off at the end?

Step 5 — Reading and commanding over MQTT

Aim. Observe the device state on MQTT topics and command it.



Publishing on .../set commands the device; subscribing to zigbee2mqtt/# shows everything

How to read the figure

The figure shows the topic tree. The device state is on `zigbee2mqtt/<name>`, and the command goes to `zigbee2mqtt/<name>/set`. The `bridge` topics describe the bridge itself.

In a terminal, subscribe to everything:

Command to run

```
mosquitto_sub -h localhost -t 'zigbee2mqtt/#' -v
```

In another terminal, command the plug:

Command to run

```
mosquitto_pub -h localhost -t zigbee2mqtt/tomada_sala/set -m '{"state": "TOGGLE"}'
```

Expected result (values may vary)

```
zigbee2mqtt/tomada_sala {"linkquality":87,"state":"ON"}
zigbee2mqtt/bridge/state online
```

Analyse

Which fields does the state message have? What is the link quality value? What would happen to the message if the device were configured with retained messages?

Step 6 — Closing and cleaning up

Confirm that join permission is turned off in the web interface. Stop the program with `Ctrl+C`. If it was installed as a service, `sudo systemctl stop zigbee2mqtt`.

Note

The `/opt/zigbee2mqtt/data` folder holds the network key and the list of paired devices. Back it up before any change, and do not publish it.

Review questions

1. Which components make up the chain between the Zigbee device and Node-RED?
2. Why should join permission be closed after pairing?
3. On which topics is state read and a command sent?
4. How does this block connect to the mini-project's middleware block?

Summary

- **Zigbee2MQTT** is an open bridge between a Zigbee network and an MQTT broker.
- **Pairing** allows entry, runs the interview and gives the device a name.
- **State** and **commands** travel on MQTT topics, accessible to any client.